

Bloco 4.7, Propostas: anexos e export XLSX — status de entrega

Data: 2026-09-08. Branch: feature-dev-breno-automacao. Working tree, sem commit.

O que o bloco é

Anexos na tela de detalhe e exportação XLSX da listagem, reusando os componentes que FD e Orçamentos já têm. As quatro atividades foram **implementadas em 2026-09-05, junto do bloco 4.5** — a aba de anexos entrou funcional em vez de placeholder, e a exportação veio no mesmo movimento. Este documento fecha o registro e traz a verificação que faltava: o upload rodando de verdade contra o Storage.

Atividade 1: aba Anexos com FileUpload, bucket e path

Concluída. `[id]/anexos-tab.tsx` sobre `FileUpload`, bucket `anexos`, path `{empresa_id}/propostas/{proposta_id}/{timestamp}_{nome}`.

O path não é escolha estética: a RLS do Storage exige que o **primeiro segmento seja o `empresa_id`** do usuário (`storage_empresa_id_from_path`, migration 003). Quem monta é o `buildStoragePath`, já existente — nada de string concatenada à mão.

O timestamp no nome evita colisão quando a mesma pessoa sobe dois arquivos com o mesmo nome, e o `sanitizeFilename` tira acento e espaço, que o Storage recusa.

Atividade 2: listar, visualizar e excluir

Concluída. A listagem mostra ícone por tipo (`iconForFile`), nome, tamanho formatado e data de envio.

Visualizar usa URL assinada de 1 hora (`createSignedUrl`), não URL pública: o bucket é privado, e link permanente num bucket privado ou não abre ou vaza. A exclusão passa por `ConfirmDialog`.

Duas ordens deliberadas, que só importam quando algo falha no meio:

1. **No upload**, o arquivo sobe primeiro e o jsonb depois. Se o update do jsonb falhar, o arquivo é removido (rollback best-effort) — sem isso ficaria órfão no bucket, invisível pela tela e ocupando espaço.
2. **Na exclusão**, o Storage vem primeiro. Falha no meio deixa o jsonb consistente com o bucket, que é o estado recuperável.

Atividade 3: server route /api/export/propostas

Concluída. No formato dos três exports que já existiam (`buildWorkbookResponse` , `BRL_FORMAT` , `DATE_FORMAT` , `todayStr`).

Duas colunas existem só na planilha, porque planilha não tem selo nem badge: **Vencida** (Sim/vazio), derivada em runtime como na tela, e **Condições**, que concatena os quatro percentuais formatados com o texto livre de `condicoes_pagamento` .

Atividade 4: ExportButton respeitando os filtros

Concluída, e é o ponto que exigia cuidado.

A rota reusa `sanitizeBusca` , `obraIdsDaBusca` e `computePeriodoCutoff` — os mesmos helpers da listagem. Se a busca fosse reescrita por fora, o XLSX não bateria com a tela de onde a pessoa clicou "Exportar", e ninguém notaria até alguém conferir número por número.

O `ExportButton` recebe `extraParams` , o mesmo objeto que alimenta a paginação, então o que está filtrado na tela é o que vai pro arquivo — inclusive `?vencidas=1` , que não é status do banco.

Verificação

CAMADA	RESULTADO
estático	<code>tsc --noEmit</code> limpo, lint sem warnings
unitário	46 casos, <code>fail 0</code>
build	31 rotas, iguais à baseline
runtime	39/39 rotas, incluindo os dois exports com asserção de conteúdo da planilha
dados	12/12 checagens sob RLS em gc-dev
escrita	35/35 passos exercitando as Server Actions, com limpeza
navegador	22/22 passos clicando na tela, com upload e remoção de anexo pelo seletor de arquivo

O upload foi exercitado de ponta a ponta pela camada de escrita: um PDF real sobe pelo `uploadAnexo` , o objeto é conferido **no bucket** (`storage.list`), o path é validado contra o formato que a policy exige, e o `deleteAnexo` remove dos dois lados. Era a pendência 3 do bloco 4.5 — a RLS de Storage sem teste.

A rota de exportação responde com `content-type` de planilha e com as células certas, verificado na camada runtime.

O conteúdo do XLSX é conferido desde 2026-09-08: a camada runtime baixa a planilha e abre com o exceljs que o projeto já tinha. Três asserções — as 12 colunas pelo nome, um mínimo de linhas de dado, e textos que precisam aparecer em alguma célula, entre eles o Sim da coluna derivada Vencida, que só sai certo se a regra de vencimento estiver aplicada na exportação também.

Testei que a asserção reprova: com uma coluna inventada, um mínimo absurdo e um texto inexistente, os três acusam. Asserção que só passa não vale nada.

Pendências

Revisadas em 2026-09-08: as duas foram resolvidas.

- 1. ~~Conteúdo do XLSX sem asserção.~~ **Resolvida.** A camada runtime abre a planilha e confere colunas, linhas e células (detalhe na verificação). O campo esperaXlsx em scripts/validacao-rotas.json serve pra qualquer export — o de obras também ganhou asserção.
- 2. ~~Nenhum arquivo real foi aberto pela tela.~~ **Resolvida.** A camada 7 agora anexa um PDF pelo <input type="file"> de verdade, via DOM.setFileInputFiles do CDP — input.files é read-only em JS, então esse é o único caminho. O roteiro sobe o arquivo, confere que ele entrou na lista salva, e remove pelo botão com o ConfirmDialog.

Duas asserções minhas estavam erradas aí, e o screenshot mostrou: o FileUpload exibe o nome do arquivo na fila de upload, com spinner, antes de o upload terminar — esperar só o nome aparecer era falso positivo. E ele mantém a fila depois do sucesso, com um "Limpar lista", então esperar o nome desaparecer após a exclusão estourava sem motivo. O que prova cada lado é a lista salva: "Nenhum anexo ainda" sair da tela no upload, e voltar na remoção.

Arquivos

ARQUIVO	O QUE É
src/app/(app)/propostas/[id]/anexos-tab.tsx	a aba: lista, visualiza e exclui
src/app/(app)/propostas/[id]/actions.ts	uploadAnexo, getAnexoUrl, deleteAnexo
src/app/api/export/propostas/route.ts	o XLSX, com os mesmos filtros da tela
src/app/(app)/propostas/page.tsx	o ExportButton com extraParams
scripts/validar-runtime.mjs	conferirXlsx : abre a planilha e confere colunas, linhas e células
scripts/navegador-cdp.mjs	anexarArquivo : DOM.setFileInputFiles , pra exercitar o seletor
scripts/validar-navegador.mjs	o upload e a remoção de anexo pela tela
docs/4.7-status-entrega.md	este documento